

Concept Note | An Open Commons for AI-Native Software Development Artifacts

Working title options: [**Agentic Delivery Commons (ADC)** | **The Field Kit**]

Status: draft for comment · License intent: Apache-2.0 (code/scripts) + CC BY 4.0 (prose artifacts) · DCO sign-off required

0. The Preface

A lot of best practices are constantly getting churned among developers as compensation for current model weaknesses. And, this is an ever evolving scenario, newer models will come forth with newer merits / de-merits, and best practices will continue to evolve alongside.

We believe the collective wisdom of the community will help converge as well as help evolve in a timely manner on these best practices. Best practices are born out of grounded nuances, varied scenarios - all the more reason to have a community play on this. Hence, we propose to create an open source repository that becomes the reflection of collective wisdom, and helps with open distribution.

1. The problem this repository exists to solve

Code generation stopped being the bottleneck. Three things replaced it:

1. **Verification cost.** AI-generated code reads as correct and still needs full scrutiny. This is being described as a verification tax, and consistently results in throughput rising while delivery stability falls. Individual output goes up; organisational delivery often doesn't move.
2. **Context fragmentation.** The knowledge an agent needs: architectural invariants, why a decision was made, what is frozen, what will page someone at 3am - is scattered across READMEs, Wikis, Slack, and people's heads. Agents don't read what they aren't pointed at.
3. **Folklore over engineering.** The practices that actually work are being rediscovered independently by thousands of teams, shared as screenshots and blog posts, and lost. Nothing accumulates.

There is now a large distribution layer for agent artifacts: public skill registries hold tens of thousands to hundreds of thousands of entries, and essentially no *curation* layer. Volume is solved. Trust, provenance, and evidence are not.

This repository is the curation layer: Not necessarily more artifacts - but better-attested ones.

2. What this is, and what it is not

It is a curated, versioned, evidence-bearing commons of artifacts that shape how AI agents participate in software delivery: context files, skills, hooks, knowledge-base structures, verification harnesses, workflow patterns, and governance instruments. Every artifact is portable, copy-pasteable, and carries a record of where it came from and what it cost or saved.

It is not:

- An awesome-list. Hundreds of collated links have a six-month life and no accountability.
- A marketplace or registry. Distribution is a solved problem; we are the thing that should sit *upstream* of a registry.
- A framework. No installer, no runtime, no lock-in. If an artifact only works inside our tooling, it doesn't belong here.

The contribution bar, stated in one line: one artifact, one scar. Every submission must name the concrete incident, review comment, outage, or repeated friction that produced it. This single rule is what keeps out LLM-generated filler, and it is the reason senior engineers will take the repository seriously.

3. The Base Thesis

Artifacts that steer AI agents are **engineering assets with a lifecycle**, not just prompts. They deserve versioning, evidence, ownership, security review, expiry, and a graveyard.

Two corollaries drive most of the design:

- Context is a scarce, contested resource.** More guidance is not better guidance. There is enough growing evidence that machine-generated context files can *reduce* task success while increasing cost, mostly by restating what the codebase already makes obvious. Every artifact here declares its token budget and obeys progressive disclosure.
- Artifacts decay as models improve.** A large share of today's "best practice" is compensation for current model weaknesses. When those weaknesses disappear, the artifact becomes dead weight that still consumes context. So artifacts declare which models they were validated against, and the repository actively retires them.

4. The Taxonomy: 7 Layers

A taxonomy is what turns a pile into a commons. Contributors need to know where their thing goes; readers need to know what's missing.

Layer	Question it answers	Illustrative Artifacts
L0 - Context	What must the agent know before it acts?	AGENTS.md patterns, steering files, glossaries, repo maps, anti-pattern registries, Architecture Design Record registries

L1 - Capability	What can the agent do well, repeatably?	Skills (SKILL.md), sub-agent definitions, MCP servers, tool contracts
L2 - Control	What must never happen, regardless of instructions?	Hooks, permission profiles, path guards, policy-as-code, sandbox profiles
L3 - Verification	How do we know the output is actually correct?	Eval suites, mutation gates, golden traces, review agents, conformance checks
L4 - Workflow	How does work flow between humans and agents?	Spec-driven flows, decomposition patterns, escalation protocols, checkpoint design
L5 - Evidence	Is any of this working?	Attribution conventions, metric definitions, anti-metrics, before/after templates
L6 - Governance	Can we defend this to security, legal, and audit?	Artifact review checklists, provenance/signing, regulatory mappings, threat models

5. Taxonomy Definition (How we can potentially organise the repository too)

L0 - Context

What does your codebase know that a competent new hire would take three months to learn?

What an agent needs to know about a codebase before it is safe to let it act. Covers how institutional knowledge: conventions, boundaries, history, the reasoning behind past decisions gets represented in a form an agent will actually consume, and how that representation stays true as the code moves. The hard constraint is that attention is finite, so this layer is as much about what to withhold as what to supply.

L1 - Capability

Which task do you still refuse to delegate, and what would have to be true for that to change?

Packaging repeatable expertise so an agent performs a task the way your best developer would, consistently, rather than the way the model's training distribution suggests. Generality should work regardless of domain, and specifics would work only inside a particular stack, regulatory regime, or risk profile. The open tension is generality versus usefulness.

L2 - Control

Where in your systems would you not accept a probabilistic guarantee, and why?

The things that must not happen regardless of what anyone human or model instructs. This layer is deterministic by design: it doesn't persuade the agent, it constrains the environment

the agent operates in. It also covers what enters the context window from untrusted sources, which is now a security boundary rather than a hygiene concern.

L3 - Verification

What convinces you a change is safe, and how much of that is still a human reading the diff?

How you establish that generated output is correct, given that fluency and correctness have decoupled. The interesting problem is that the usual quality signals were designed to measure human work and can now be satisfied without the underlying property holding. This is where most of the cost and risks have migrated.

L4 - Workflow

Where does your process break specifically because the work is being done by an agent rather than a person?

How work is decomposed and how humans and agents hand off to each other. Includes when rigour is worth its cost and when it is ceremony, how parallel work avoids collision, and where a human checkpoint adds signal versus where it degrades into approval theatre. Almost all published guidance here quietly assumes new code on a clean slate.

L5 - Evidence

What number are you being asked to report, and do you believe it?

Whether any of this is working, and how you would know. Requires separating assisted work from unassisted work at a granularity that permits comparison, and choosing measures that resist being gamed by the very acceleration they're meant to assess. Equally important is naming the measures that look like progress and aren't.

L6 - Governance

What would stop this from being adopted in your organisation, independent of whether it works?

What you would need to show a security reviewer, an auditor, or a regulator. Covers where an artifact came from, who is accountable for it, what it is permitted to do, and what obligations attach to code whose provenance is partly machine. The trust model here is closer to package management than documentation, and the ecosystem is currently in a pre-review era.

6. Why a senior engineer should spend a Saturday on this

Plainly: most of what you have learned about making agents useful currently lives in a private repository, will not survive your next job change, and is being independently rediscovered by ten thousand other people this month. There is nowhere credible to put it. Registries will take it but won't review it; blog posts get read once; your employer's wiki dies with the reorg.

This is an attempt to build the place where that knowledge accumulates with attribution, review, and an honest expiry date attached.=
